

Pipeline Validation: POS-Tagger Migration and Associated Fixes

TODO: author name

September 27, 2026

Abstract

This note documents a validation pass performed on the Macedonian-language stylometric pipeline after its part-of-speech tagger was migrated from spaCy (`mk_core_news_lg`) to CLASSLA-Stanza. It covers (i) how much the downstream Louvain community structure on the semantic knowledge graph (SKG) changed as a result of the tagger switch, separated from ordinary seed-to-seed algorithmic noise, including the diagnosis and subsequent fix of a community-detection artifact (author-similarity edges alone defining a 13-author community with almost no word support) and the seed-stability tradeoff that fix introduces; (ii) two correctness bugs found and fixed during the pass, one in rhyme-ending extraction and one in the style-transfer candidate-selection fallback tier; (iii) two metric additions — a length-normalized lexical-diversity measure and stanza-level structural features — added to the per-author style profiles; and (iv) §7, a later addition, the first quantitative evaluation of the distilled Qwen3 LoRA style-transfer model against its own untuned base model and against the Gemma teacher that produced its training data, which finds that fine-tuning succeeded on every axis the task is defined on but collapsed ten target voices into one, and separates that collapse from a repetition artifact that turns out to be purely a decoding setting.

1 Background: the tagger migration

The pipeline originally tagged the corpus (2,711 Macedonian poems and song lyrics, 24 authors after the ≥ 5 -poem eligibility filter) with spaCy’s `mk_core_news_lg` model. This was replaced with CLASSLA-Stanza, which lemmatizes inflection correctly and additionally exposes MULTEXT-East `xpos/feats` morphology, enabling the `feats`-keyed morphological lookup used by style transfer. The full pipeline — POS tagging, TF-IDF profiling, SKG construction and enrichment, embeddings, and graph analytics — was rerun end to end against the new tags. This note asks: how much did that rebuild actually change the pipeline’s outputs, and were any of those changes symptoms of pre-existing bugs rather than the tagger switch itself?

2 Louvain community stability under the tagger switch

2.1 Setup

Author-similarity communities are computed by `graph_analytics.py` via `community.best_partition(G, weight='weight', random_state=42)` on the SKG (`skg_final.gexf`). Both the pre-switch (spaCy) and post-switch (CLASSLA) versions of `graph_analytics.py` call Louvain with these exact same parameters, so a direct diff of the two partitions is a fair comparison, not confounded by a methodology change alongside the tagger change. The pre-switch graph and analytics output were recoverable from git history (git commit 5f03983, carried forward unchanged by the pure file-reorganization commit 363009a); the post-switch version is the current repository state (commit 41bb49e onward).

2.2 Direct diff against the pre-switch partition

Both partitions cover the same 24 authors and both settle on 6 communities. Comparing author community labels directly (after optimal label alignment) gives:

Metric	Value
Adjusted Rand Index (ARI)	0.221
Normalized Mutual Information (NMI)	0.483

An ARI of 1 would mean the partitions are identical (up to relabeling); an ARI near 0 means the agreement is no better than chance. 0.221 indicates a substantial reshuffling — but Louvain is itself stochastic in the order it visits nodes, so this number alone cannot distinguish “the tagger switch changed the graph structure” from “Louvain would have landed somewhere this different anyway, even on an unchanged graph, under a different run.”

2.3 Establishing a noise floor

To separate the two, Louvain was re-run 30 times on the *current* (post-switch) graph with `random_state` $\in \{0, \dots, 29\}$, holding the graph itself fixed. Each run was compared to the documented `random_state=42` reference partition:

Comparison	Min	Mean	Median
Each of 30 seeds vs. the seed = 42 reference (ARI)	0.515	0.877	0.906
All pairs among the 30 seeds (ARI, 60 sampled pairs)	0.469	0.819	—

The number of communities Louvain settles on also varies by seed alone (4, 5, or 6 communities across the 30 runs).

2.4 Conclusion: a real change, not just seed noise

Seed-only variability on a fixed graph floors at $\text{ARI} \approx 0.47$ –0.5 in the worst observed case. The pre/post-switch ARI (0.221) sits *below* that floor. The tagger switch therefore produced a partition shift larger than ordinary Louvain run-to-run noise would account for — a genuine, if partial, structural change, not an artifact of re-running the algorithm once.

The change is concentrated among the “word-driven” authors (see §2.5 for the exception). Three of the six pre-switch communities (covering 14 of the 24 authors) dissolved and redistributed across different post-switch communities — e.g. the pre-switch cluster {Рацин, Поповски, Коцев, Линдро, Лазаров} split into four different post-switch communities. This is the expected effect of a tagger change: different lemmatization directly perturbs the TF-IDF and co-occurrence edges those communities are built from.

2.5 Diagnosis: Community 0, a stable but low-information clique

The current partition’s largest community (“Community 0”) groups 13 of the 24 authors around only 2 surviving word nodes:

Петре М. Андреевски, Влада Урошевиќ, Блаже Конески, Славко Јаневски, Ацо Шопов, Матеја Матевски, Богомил Ѓузел, Адем Гајтани, Веле Смилевски, Пијан Славеј, Ефтим Клетников, Ристо Лазаров, Ива Петреска

Seed stability. All $\binom{13}{2} = 78$ pairs of these authors co-clustered together in *every one* of the 30 seed runs described above — the single most reproducible feature of the entire partition, far above the general seed-noise floor.

Why it forms. Breaking down each author’s SKG weighted degree by edge type shows why: every Community-0 author draws 40–70% of its weighted degree from direct `author_similarity` edges (author $\leftrightarrow$ author Jaccard-vocabulary-overlap edges), versus 86–100% from word-mediated (`distinctive/cooccurrence`) edges for every author outside the community. Community 0 is not a set of authors Louvain found to write alike through shared vocabulary; it is the set of authors whose word nodes did not survive strongly into the graph (consistent with the SKG’s frequency floors and node cap), leaving the direct author-similarity edges as the dominant signal available to place them by.

Not new to the tagger switch. The same phenomenon already existed pre-switch, with a 9-author core — Андреевски, Урошевиќ, Конески, Јаневски, Ѓузел, Гајтани, Славеј, Клетников, Петреска — all a subset of today’s 13, each already showing 13–47% author-similarity weight share before the tagger changed. The switch grew this core from 9 to 13 authors (adding Шопов, Матевски, Смилевски, Лазаров), it did not create it.

Interpretation. Community 0 should be read as “authors the graph has too little word-level signal to place elsewhere,” not as evidence of shared literary style — and, as described next, was subsequently removed by changing what Louvain is allowed to see.

2.6 Fix: excluding `author_similarity` edges from the partitioning graph

`graph_analytics.py` now runs Louvain on a copy of the SKG with every `author_similarity` edge removed, keeping `cooccurrence`, `semantic_similarity`, and `distinctive` edges — i.e. only edges mediated by an actual word — as the partitioning signal. `author_similarity` edges are still computed and still present in `skg_final.gexf` for every other purpose (PageRank, weighted degree, visualization); they are excluded only from the graph Louvain sees, since §2.5 showed they were sufficient on their own to define a community with no word-level support behind it. The direct author-similarity signal remains available where it was always meant to be used as an auxiliary/fallback source — in `candidate_selection.author_similarity()`, which already computed it independently of the graph, for `tier_cross_author_similar`.

Result. Community 0 no longer exists. Louvain on the restricted graph settles on 5 communities instead of 6, and every one now has substantial word support (49–102 word nodes each, versus the old Community 0’s 2). The 13 authors previously grouped by author-similarity edges alone are now distributed across all 5 communities on genuine vocabulary evidence:

Former Community-0 author	New community (by word evidence)
Блаже Конески, Славко Јаневски, Ацо Шопов	Community 2 (with Кочо Рацин)
Влада Урошевиќ, Пијан Славеј, Ефтим Клетников	Community 1
Петре М. Андреевски, Богомил Ѓузел, Ристо Лазаров, Ива Петреска	Community 3
Адем Гајтани, Веле Смилевски	Community 4
Матеја Матевски	Community 0 (new, unrelated to the old one)

Cost: the new partition is less seed-stable. Repeating the 30-seed sweep from §2 on the restricted graph gives a markedly lower stability floor than the full graph had:

Graph	Min ARI	Mean ARI	Median ARI
Full graph (incl. <code>author_similarity</code>), §2	0.515	0.877	0.906
Restricted graph (word-mediated only)	0.300	0.581	0.541

This is the expected price of the fix, not a sign it is wrong: the author-similarity edges were direct, high-weight author↔author connections that gave Louvain unambiguous structure to lock onto: precisely the “glue” that made Community 0 so artificially stable (§2.5). Without it, the remaining word-mediated graph is sparser, and Louvain’s well-known sensitivity to node-visit order shows up more. Per-community confidence is uneven: averaging pairwise co-clustering across the 30 seeds by the seed = 42 community gives 0.88 for Community 4 and 0.74 for Community 3 (high confidence) down to 0.49 for Community 2 and 0.27 for the 3-author Community 0 (Матевски, Линдро, Цветкоски — low confidence, on par with a coin flip for some pairs).

Recommendation. Treat the 5-community, word-mediated partition as the current best estimate of stylistic community structure, but do not read the exact `random_state` = 42 assignment as ground truth for the weaker communities (2 and the new 0) the way one might have been tempted to for the old, artifactually-stable Community 0. A seed-ensemble/consensus partition would give a more defensible per-pair confidence than any single seed and is a natural next step if community membership starts being used for a downstream decision (e.g. cluster-based candidate pooling) rather than reported descriptively.

Addendum (poem-id migration). The table above and its seed=42 assignments were computed before a later fix gave every poem a stable `poem_id`, correcting ~15 (`author`, `song_title`) pairs in the corpus that had been sharing a title across distinct physical poems (see `CLAUDE.md`’s Data quality context). That fix nudges `pos_transitions.json` and the SKG’s edge weights by a small, legitimate amount for the affected authors — among them Блаже Конески, Ефтим Клетников, Влада Урошевиќ, and Пијан Славеј. Re-running `graph_analytics.py` after the fix shows this small a perturbation is enough to move Конески out of his former group with Јаневски/Шопов/Рацин and into the group with Гајтани/Смилевски instead, while Урошевиќ/Славеј/ Клетников and Андреевски/Ѓузел/Лазаров/Петреска remain grouped together (under new community numbers — Louvain’s own labels are not stable across runs and carry no meaning by themselves). This is not a contradiction of this section’s finding; it is a further, unplanned demonstration of it: this restricted graph’s community structure is sensitive enough that even a small, targeted correctness fix elsewhere in the pipeline reshuffles the weaker communities. Properly updating the table above would mean repeating the full 30-seed sweep on the corrected graph, not just swapping in one new run’s numbers — that re-verification is out of scope for the poem-id migration and is a natural next step before this section’s exact figures are relied on again.

3 Correctness fixes found during validation

Two bugs, unrelated to the tagger switch, were found and fixed while validating pipeline outputs.

3.1 Rhyme-ending extraction

`style_profiler.py`’s `top_rhyme_endings` field took the literal last three characters of each line’s last whitespace-split token, with no punctuation stripping. Macedonian poetry in this corpus frequently ends

lines on a standalone em/en dash (a caesura marker) or on punctuation glued to the last word, so every one of the 24 authors’ top-5 rhyme endings were contaminated with punctuation artifacts — e.g. Блаже Конески’s were -- | на, | на. | ам. | ат. and Кочо Рацин’s were -- | --- | ата | ите | ни, . This was corpus-wide, not an isolated case.

The fix extracts a genuine line-final syllable: it walks backward past trailing pure-punctuation tokens, strips edge punctuation from the real last word, and returns the substring from the onset of the final vowel’s syllable (the character after the second-to-last vowel) through the end of the word. After the fix, no author’s top rhyme endings contain punctuation, and the endings reflect real Macedonian grammatical suffixes (-та, -те, -на, -ва, and similar).

3.2 Style-transfer legacy-fallback tier

candidate_selection.py’s last-resort candidate tier (tier_legacy_fallback) imported a hardcoded TARGET_AUTHOR constant from llm_probe.py instead of using the target_author parameter it was already given, so whenever this tier fired for any author other than that hardcoded default, it silently supplied *that other author’s* TF-IDF vocabulary as replacement candidates. The fix threads target_author through to load_target_words(), which now accepts an explicit author rather than reading the module constant.

To measure real-world impact, style_transfer.transfer_style() was re-run over 960 sampled (source poem, target author) pairs spanning all 24 authors, instrumented to record which tier produced each of the 20,940 replacement slots evaluated:

Tier of the candidate chosen in the output	Share of slots
primary	94.20%
relaxed_semantic	5.73%
legacy_fallback	0.062% (13 slots)
cross_author_similar	0.014%

The bug was real wherever it fired, but it fired rarely — 13 of 20,940 slots, concentrated in a handful of sparse-vocabulary authors (8 of the 13 for a single author, Јосип Коцев). It was corrupting a rare edge case, not a meaningful share of style-transfer output.

4 Metric additions to the author style profiles

4.1 Length-normalized lexical diversity

Plain type-token ratio (TTR) in author_style_profiles.csv is not length-normalized, and per-author corpus size varies 37-fold across the 24 authors (232 to 8,538 content tokens). On this corpus, plain TTR correlates with token count at Pearson $r = -0.76$ ($p = 1.7 \times 10^{-5}$) — the expected artifact of TTR mechanically falling as a corpus grows, rather than a signal of authorial lexical richness. A Moving-Average Type-Token Ratio (MATTR; Covington and McFall, 2010) column was added, averaging the TTR of every 200-token sliding window over each author’s lemma sequence (window size chosen to fit inside the smallest eligible author’s 232-token corpus). MATTR against token count: $r = -0.037$ ($p = 0.86$) — no residual length dependence. The original type_token_ratio column is retained for reference but is now documented as diagnostic-only; mattr is the column to use for cross-author comparison.

4.2 Stanza-level structure

`author_style_profiles.csv` previously had no stanza-level features. Poem text in `stripped_songs.csv` uses blank-line-delimited stanzas (the same convention already relied on elsewhere in the pipeline for stanza extraction). Two fields were added, computed per author from that delimiting: `avg_stanzas_per_poem` and `avg_lines_per_stanza`. Values are plausible and vary meaningfully across authors — e.g. Ристо Лазаров averages 7.23 stanzas/poem at 2.95 lines/stanza (short, couplet-like stanzas), while Лилјана Дирјан averages 1.14 stanzas/poem at 15.62 lines/stanza (long, largely unbroken poems).

5 Worked example: style transfer after the fixes

As a concrete illustration of where the pipeline stands after the fixes in this note, this section walks through one full style-transfer run on the current (post-fix) pipeline state: Кочо Рацин’s “Балада за непознатиот” (“Ballad of the Unknown [Soldier]”), rewritten in Блаже Конески’s style via `style_transfer.transfer_style(text, ‘Блаже Конески’)`. This is a real, reproducible pipeline run, not a hand-picked or edited example — it was independently regenerated against the current corpus and model files and diffs byte-identical (modulo typographic quote-mark style) to the run quoted below.

```
Натаму – в поле битолско
чемрее врба проклета –
под врбата незнаен гроб,
в гроб лежи војник непознат.

Лежи од војна света,
лежи – и веќе земјосал –
силно го тага гласи
мрзне тука загинал.

Никој крај него немаше –
вишното небе врз него,
земјата скришна под него,
над гробот врба спремам.

(...)

И од ним – самовилите
една по една идеа –
од гроб до гроб го мрзне
јунак до јунак – на оро.

(...)
```

(Full output omitted here for space; see the corpus entry for the source text and re-run `style_transfer.transfer_s` for the complete transferred poem.)

What this demonstrates, and what it doesn’t. This call did not pass `source_author`, so only replacement criterion (b) — a lemma absent from the target’s vocabulary for a POS in `B_ELIGIBLE_POS = {ADJ, ADV, VERB}` — is active; criterion (a) (source-distinctive/target-absent, any content POS) needs a source author and did not fire here. This is visible directly in the output: NOUN tokens like `гроб` (‘grave’) and `гробишта` (‘graveyard’) are preserved verbatim throughout, exactly as `style_transfer.py`’s own design comment (labelled “Criterion (b)”) intends — nouns carry subject matter, not style, and are deliberately left alone under this criterion. Adjectives and verbs are substituted where Конески’s own vocabulary lacks the Рацин original: `војна световна` (‘worldly/mundane war’) becomes `војна света` (‘holy war’), and `тага изела` (‘grief consumed [him]’) becomes `тага гласи` (‘grief speaks/sounds’), each drawing a real Конески-attested word of the same POS into the same grammatical slot.

Two honest limitations visible in this same output. First, `задека` ('because') — a conjunction — is itself replaced (with `мрзне`, 'freezes'), which looks wrong until you check the tagging: CLASSLA tags this token VERB in this context (Aspect=Perf|Tense=Past), not CONJ/SCONJ, so it passes through `CONTENT_POS` and becomes a legitimate replacement target under the (mistaken) tag. This is the same category of tagging error `pos_flags.csv`'s `upos_xpos_clash/unstable_` tag reasons are built to catch for common cases — `задека` was not one already in `data/pos_corrections.csv` at the time of this run, so it slipped through uncorrected. Second, `дигаа` ('they raised', Number=Plur, 3rd-person imperfect) is replaced by `мрзне`, which surfaces in an unmarked/singular-present-looking form — the morphological lookup evidently had no attested Конески plural-imperfect form of that lemma to realise into, and fell back to a lower morph tier (`nearest_attested` or `unresolved`) rather than the requested agreement. Neither of these breaks the pipeline, and neither is new to this session's fixes — they are exactly the kind of residual, low-frequency noise the tier/morph-tier logging in `candidate_selection.py` exists to make visible and auditable, rather than silent.

6 Prompt-construction fixes in the LLM style-transfer pipeline

A second, independent style-transfer approach (`llm_style_transfer.py`) sends a single LLM prompt — an LLM-written summary of the source poem, its content keywords, the target author's style profile in prose, a relevance-filtered vocabulary palette, tone exemplar lines, and structural targets — and asks the model to write a new poem, rather than substituting tokens into masked slots. Auditing the per-run records it writes to `data/llm_transfer_logs/` surfaced three defects in how that prompt was assembled. None of them raised an error; all of them degraded the prompt silently.

6.1 Duplicate exemplar lines

The tone-exemplar block repeated lines verbatim. In a Ацо Шопов run, four of nine exemplars were two duplicated pairs (`Треба да бидеме подобри.` and `Вечер е тива, дремлива,`), and a further two came from a single poem. The block is introduced to the model as a rhythm reference, so a verbatim repeat inside it is an instruction to repeat; the generated poem duplicated whole stanzas and repeated single lines three times.

Duplicates entered by two routes. First, `exemplar_selection.py`'s `qualifying_lines()` capped candidates at two lines per poem but did not check whether they were the same text, so a refrain spent the entire quota on one line — `Треба да бидеме подобри` opens with that line twice (`поем_id 88`), and `Партизанска колона` repeats `Вечер е тива, дремлива,` at lines 2 and 21 (`поем_id 50`). Because identical text yields an identical line embedding, both copies always landed in the same k-means cluster and the two-per-cluster rule selected both. Second, the corpus's near-duplicate poems each contributed the same line independently: `Блаже Конески`'s `Бура` (`поем_id 13`) and `БУРА` (`поем_id 566`) are one poem under two title casings, and 24 such author/lowercased-title groups exist.

The fix de-duplicates normalized line text (case-folded, whitespace-collapsed) author-wide rather than per-poem, which closes both routes; a repeat is skipped without consuming a per-poem slot, so a refrain poem still contributes two distinct lines. Separately, `cluster_and_select()` now enforces one line per source poem globally across clusters rather than within them, and processes clusters smallest-first so that a constrained cluster claims its poem before a roomier one takes it. Across the corpus's 38 authors this costs 7 exemplar lines in total (218 to 211) and no author falls below 5. Exemplar records now carry `поем_id` alongside `song_title`, since the title alone cannot distinguish `Бура` from `БУРА` in a run log.

6.2 Summary language

Summaries were requested in Macedonian and returned ungrammatical: the logged summary for Кочо Рацин’s Балада за непознатиот opened Песната приказува (for приказува), and continued го тага неговото загижување and го поседуваат несигурниот смисол на нивната смрт — text that conveys the gist but is not well-formed Macedonian. The local model in use (qwen3:14b) reads Macedonian considerably better than it writes it, and the summary is scaffolding consumed by the generation step, not output shown to a reader.

Summaries are therefore now taken in English. The prompt’s lexical material — content keywords, vocabulary palette, exemplar lines — and the generated poem itself remain Macedonian; only the description of the source poem’s content changes language. This matches the existing choice to phrase instructions in English (DEFAULT_INSTR_LANG). SUMMARY_PROMPT_VERSION was bumped to 2, which invalidates cached v1 summaries via the cache key.

6.3 Unstated output language

The prompt never instructed the model which language to write in. Under v1 this went unnoticed because the Macedonian summary was the largest Cyrillic block in the prompt and the model followed it by example. Moving the summary to English removed that implicit cue, so the instruction is now explicit (*“Write the poem in Macedonian, in Cyrillic script”*). The same audit found that `--lang mk` was an advertised CLI choice with no corresponding entry in PROMPT_TEMPLATE, so it raised `KeyError` during prompt assembly; the choices are now derived from the templates that actually exist.

6.4 A lemmatization error surfaced by the prompt

Reading the assembled prompts also caught a tagging error that no flag rule raises. The content keywords for Кочо Рацин’s Балада за непознатиот included петлин, which is not a Macedonian word: classla lemmatizes the definite plural петлите to петлин rather than петел (‘rooster’). It gets every other form of the word right — петел, петелот and петли all lemmatize to петел — so the corpus carried one lemma split across two spellings, 6 tokens under the wrong one, across three authors (Кочо Рацин, Милош Линдро, Ефтим Клетников).

Nothing in `pos_flags.csv` catches this: the rules look at tags, not at lemma consistency, and the POS and `xpos` (Ncmpny) were both correct here. The fix is a lemma-only entry in `data/pos_corrections.csv` — blank correction, lemma_fix of петел, corpus-wide scope — which `corrections.apply_to` applies without clearing `xpos/feats`, since no POS judgement was overruled.

The effect on the prompt is not simply a substituted keyword. Merging the 6 tokens into петел’s existing count raises that lemma’s corpus frequency, lowering its IDF, and it consequently drops out of the poem’s top-10 keywords altogether — the previous ranking was inflated by a spurious singleton lemma. Lemma-keyed artifacts further downstream (`word_embeddings.json`, the SKG, `morph_lookup.json`) still carry a stale петлин entry; at 6 tokens in 70,531 it was not judged worth a full rebuild, and петел already has an embedding, so the keyword remains embeddable.

6.5 Switching the generation model

All of the prompt work above was developed and logged against `qwen3:14b` running locally under Ollama. With the prompt defects fixed, that model was the remaining constraint on output quality. It writes Macedonian poorly: generated poems repeated whole stanzas, ran single lines three times (и се вика: ``Иде, иде, иде, иде, иде.''), and produced non-words and broken agreement (кое се вик, каде се вод).

The pipeline was therefore switched to `gemma-4-31b-it`, a substantially larger model with much stronger Macedonian. It is now the default (`DEFAULT_MODEL/DEFAULT_BACKEND` in `llm_style_transfer.py`); Ollama and `qwen3:14b` remain reachable through `--backend ollama --model qwen3:14b`, so the earlier logs stay reproducible.

Two practical obstacles shaped how it is reached. First, every OpenRouter :free Gemma route answers HTTP 429 from a shared upstream pool at Google AI Studio, so the model is called through Google’s Generative Language API directly, with its own key — a third `google` backend in `llm_client.py` alongside `ollama` and `openrouter`. Second, Gemma 4 always reasons before answering, and cannot be stopped from doing so: `generationConfig.thinkingConfig.thinkingBudget` is rejected with 400 `Thinking budget is not supported for this model`. The scratchpad returns as a response part flagged `thought=true` beside the answer part, so it has to be filtered out — concatenating every part yields the model arguing with itself rather than the poem.

Thinking tokens are charged against `maxOutputTokens`, and their number varies enough between runs of the *same* prompt that no fixed budget is safe: observed figures for one style-transfer prompt were 3231 thought tokens (305 for the poem), then 15,997 with no answer produced at all, then 2075 on a re-send. The backend therefore requests the model’s full 32,768-token output ceiling and retries once when a response comes back with thinking but no answer part, rather than attempting to budget around the variance.

6.6 Effect

Re-running both logged transfers from the same source poem confirmed the prompt-level fixes: exemplar blocks now contain no repeated line and no repeated source poem (8 lines/8 poems for Ацо Шопов, 10/10 for Блаже Конески), and the summary is fluent, accurate English.

The model switch is visible in the structural fit, which was measured but never targeted by either change:

Structural target (same prompt, same source poem)	<code>qwen3:14b</code>	<code>gemma-4-31b-it</code>
Ацо Шопов, lines (target 20.7)	16	22
Ацо Шопов, Δ tokens per line	+0.99	+0.10
Блаже Конески, lines (target 16.4)	14	16
Блаже Конески, stanzas (target 2.3)	2	2

The qualitative gap is larger than the table suggests. Gemma places the target author’s palette words in grammatical context rather than stacking them — `ТИШИНАТА ЛИЧИ НА ТЕШКА НЕБИДНИНА` uses Шопов’s own coinage `небиднина` in a well-formed clause — and for Конески it produced a strict 8 + 8 poem built on a consistent adjective–adverb–verb line template (`студена коска тесно го стега, чемреена мака бавно го јаде`), which is the adjective-before-noun pattern that author’s style profile describes.

Content preservation improved as well, and traceably so: the English summary identifies that the dead soldier *refuses* to join the heroes’ *oro* because he lacks their honour. That refusal is the source poem’s central turn, it is absent from the Macedonian v1 summary, and it appears in both regenerated poems.

7 Evaluating the distilled SFT model against its base and its teacher

The Qwen3 LoRA pipeline distills the heavily scaffolded Gemma prompt (style profile prose, relevance-filtered vocabulary palette, exemplar lines, explicit structural targets) into a 4B model that is given only a source poem and a target author’s name. This section reports the first quantitative evaluation of that distillation: the tuned adapter against the untuned base model on identical inputs, read against the Gemma teacher that produced its training data and against the target authors’ real poems.

7.1 Setup

Both models generated over the same 468 (source poem, target author) pairs: 50 poems sampled from the `val` split, crossed with the 10 best-profiled target authors, self-transfer pairs skipped. Identical prompt format (`sft_data.render_inference_prompt`), identical decoding (sampling, $T = 0.7$, $\text{top-}p = 0.9$, 512 new tokens) and identical per-item seeds. Both runs were executed back to back in one Slurm job on one GPU (`slurm/compare_base_lora.slurm`): sampling draws from the device’s own RNG stream, so a CPU run compared against a GPU run would confound the adapter’s effect with a change of generator.

Scoring is `src/style_metrics.py`, applied by `src/compare_generations.py`. Every metric is oriented so that higher is better, because the same functions are intended to serve unchanged as reward components in the GRPO stage; a reward that disagreed with the evaluation metric would be a defect rather than a design choice.

Two metric-design points are worth stating explicitly, because both were forced by observed behaviour rather than anticipated.

line_novelty, or why copying had to be detected. The untuned base model’s dominant response is not a failed style transfer. It is the source poem reproduced verbatim, under a markdown heading, followed by an English or Macedonian paragraph explaining the target author’s style. That output attains near-perfect content preservation and — being real human poetry — excellent lexical diversity and fluency. Under any combination of diversity, fluency and content metrics, “change nothing” would therefore have been the highest-scoring strategy. `line_novelty` is the guard: the share of generated lines that are *not* verbatim source lines, after normalizing away punctuation and case.

The classifier anchor is not 1.0. Target-style match is scored with `models/classifier.pkl`, the character-(2,4)-gram author classifier. Scored on the target authors’ real poems it returns top-1 accuracy of 1.000, but this is memorization: `classifier.py` fits on the entire corpus, so the shipped model has seen every poem it is being asked to grade. The honest ceiling is the same recipe’s held-out accuracy under 5-fold cross-validation, 0.883 on these 10 authors. That figure, not 1.0, is what a real poem scores when the classifier has not memorized it.

7.2 Result

Metric (higher is better)	Base Qwen3-4B	LoRA-SFT (tuned)	Gemma teacher	Real poems
<i>Task compliance</i>				
line novelty (not copied from source)	0.224	1.000	1.000	—
no markdown / commentary	0.592	1.000	1.000	0.960
structural fit to target	0.680	0.848	0.950	0.768
<i>Target-style match</i>				
classifier top-1 = target author	0.009	0.141	0.589	0.883 [†]
classifier probability of target	0.024	0.115	0.381	—
target TF-IDF vocabulary hit rate	0.005	0.044	0.311 [‡]	0.050
<i>Content preservation</i>				
content-word recall from source	0.901 [§]	0.173	0.176	—
<i>Degeneracy</i>				
distinct-1	0.635 [§]	0.515	0.768	0.770
line uniqueness	0.895	0.904	0.999	0.969
degenerate outputs (% of rows)	18.6%	25.0%	0.2%	6.6%

[†] held-out 5-fold CV, not the memorizing shipped model. [‡] the teacher was *handed* this palette in its prompt, so it exceeds human text here; the metric measures palette uptake, not quality. [§] inflated by verbatim copying — see below. “Degenerate” is distinct-1 < 0.40 or line uniqueness < 0.80.

Fine-tuning worked, on every axis the task is defined on. The tuned model writes original lines instead of copying (1.000 vs. 0.224), never emits markdown scaffolding or commentary (1.000 vs. 0.592), fits the target author’s structure far better (0.848 vs. 0.680), and is 15 times more likely to be identified as the requested author by an independent stylometric classifier (0.141 vs. 0.009). Its content-word recall, 0.173, matches the teacher’s 0.176 almost exactly: the degree of content preservation was distilled faithfully.

The base model’s apparent diversity advantage is an artifact. Read naively, the base model wins on distinct-1 (0.635 vs. 0.515). It does not. 87.8% of its outputs are mostly copied from the source poem, and it is the copied human poetry that carries the diversity. Restricted to the 12.2% of rows where the base actually composes something new, its distinct-1 falls to 0.532 — statistically indistinguishable from the tuned model’s 0.515, and accompanied by a classifier top-1 of 0.035. Fine-tuning did not cost diversity; it removed the copying that was disguising the base model’s own lack of it.

7.3 The real deficit: style collapse, not repetition

The repetition visible on inspection is real — 25.0% of tuned outputs are degenerate by the threshold above, against 0.2% for the teacher — but it is a symptom of a larger failure that the aggregate table hides. Broken out per target author, the tuned model does not have ten voices:

Target author	Base	LoRA-SFT	Gemma teacher
Блаже Конески	0.000	0.574	0.899
Пијан Славеј	0.000	0.395	0.565
Ефтим Клетников	0.021	0.170	0.655
Ристо Лазаров	0.000	0.106	0.608
Веле Смилевски	0.000	0.104	0.881
Братислав Ташковски	0.000	0.022	0.579
Влада Урошевиќ	0.000	0.021	0.816
Адем Гајтани	0.000	0.021	0.345
Славко Јаневски	0.000	0.020	0.183
Катица Кулавакова	0.065	0.000	0.395

Classifier top-1 accuracy, per requested target author. The tuned model’s entire aggregate score is carried by two authors, and those two — Блаже Конески (182 poems) and Пијан Славеј (160 poems) — are simply the two largest authors in the corpus. Asked for Катица Кулавакова it succeeds zero times in 46. Regardless of which author is requested, the classifier reads 35.0% of its output as Блаже Конески. The teacher, by contrast, is differentiated across the whole range (0.183–0.899).

The model has learned one averaged poem-voice, not ten. Pooled vocabulary statistics at a matched 30,000-token budget confirm this is corpus-level collapse rather than per-poem repetition: 3,087 distinct word types for the tuned model, against 5,376 for the teacher and 9,060 for real poems by the same authors.

Inspection shows the collapse is syntactic as well as lexical: the tuned model falls into a fixed line template, ⟨noun⟩ ⟨adverb⟩ ⟨verb⟩ *ceza*, and fills it. This is visible even in its *best*-scoring outputs, not only its worst; its highest classifier probability on the entire run (0.785, for Пијан Славеј) is a poem in which the line Мераци брзо се вртат сега occurs four times.

7.4 Separating degenerate decoding from a degenerate policy

The repetition and the style collapse could in principle be one fault or two. A decoding ablation separates them: the same adapter, the same 56 validation pairs and the same seeds, under four decoding settings (`slurm/decoding_ablation.slurm`).

Metric	Plain sampling	Repetition penalty 1.1	Repetition penalty 1.2	No repeated 4-gram
distinct-1	0.503	0.807	0.895	0.751
line uniqueness	0.876	0.998	1.000	1.000
degenerate outputs	30.4%	0.0%	0.0%	0.0%
classifier top-1 = target	0.125	0.107	0.161	0.125
paired change vs. plain	—	−0.018	+0.036	+0.000
content-word recall	0.125	0.078	0.031	0.088
structural fit	0.859	0.837	0.813	0.837

The two faults are independent, and only one of them is a training problem.

The repetition is a decoding artifact. A repetition penalty of 1.1 — a single generation keyword, no retraining — eliminates degenerate output entirely (30.4% to 0.0%) and lifts distinct-1 from 0.503 to 0.807, past both the Gemma teacher (0.768) and the target authors’ own poems (0.770). Banning repeated 4-grams

achieves the same at 0.751, closest to the human figure. The looping was never in the model’s distribution in a way that required more training to remove.

The style collapse is not. None of the three settings moves target-style match: paired per-item changes in classifier top-1 are -0.018 , $+0.036$ and $+0.000$ on $n = 56$, i.e. noise around zero. Making the model stop repeating itself does not make it write in the requested author’s voice. That deficit is in the policy, and only training can move it.

Cost of the fix. Both settings trade some content preservation for the diversity: content-word recall falls from 0.125 to 0.078 (penalty 1.1) and to 0.031 (penalty 1.2), because a repetition penalty cannot distinguish a word looped by the model from a word legitimately carried over from the source poem. Banning repeated 4-grams is the better trade here (0.088), since it constrains sequences rather than individual tokens — at the cost of also forbidding a deliberate refrain, which real poems in this corpus do use (their line uniqueness is 0.969, not 1.000).

7.5 Two defects found during the evaluation

Validation generation ran on the CPU. `AutoModelForCausalLM.from_pretrained` places the model on the CPU unless told otherwise, and `generate()` then runs there silently, with no error and no warning. The first validation run therefore took 131 s per generation — 17 hours for 468 rows — on a node holding an idle A100. The same work takes 15.5 s per generation once the model is moved to the device, a factor of 8.5. `load_adapter_model` now logs the device it actually loaded onto, so the next occurrence is visible in the job log rather than in the wall clock.

6.2% of training examples were silently truncated. Training used `max_seq_length = 1024`. Measured against the Qwen3 tokenizer, the 1,146 synthetic training examples have a median length of 622 tokens but a 99th percentile of 2,364 and a maximum of 4,031: 71 examples (6.2%) exceed the limit and are cut. A truncated example loses the end of its assistant turn, and with it the end-of-turn token — so with `assistant_only_loss`, those examples train the model to continue past where a poem should have stopped. This is a plausible contributor to the observed looping, and is separable from the style collapse. `max_seq_length = 2048` would cover approximately 99% of examples.

7.6 Implications for the next stage

The evaluation splits the remaining gap to the teacher into three parts, and each needs a different fix.

- *Repetition* needs a change to decoding, not retraining. The reported configuration for the tuned model should include a repetition control, and the full 468-pair comparison should be rerun under it so the headline table reflects the decoding that would actually ship. A penalty of 1.1 or a 4-gram ban both qualify. The ban costs less content recall, and the penalty keeps deliberate refrains possible.
- *Truncated training examples* are a data-pipeline fix: retrain at `max_seq_length = 2048` so that every assistant turn ends with its end-of-turn token.
- *Style collapse* is what the GRPO stage exists to fix. Supervised distillation has taught the task format, but not the separation between authors, and its per-author accuracy tracks corpus size. The style-match reward should therefore be computed per target author, or balanced across authors, so that a policy which reverts to Блаже Конески is not rewarded for it. Content-word recall has to be a reward term in its own right, because the teacher preserves little content (0.176), so imitating it alone cannot raise recall.

8 Reward-driven fine-tuning (GRPO): reward design and validation

The second training stage fine-tunes the distilled model with GRPO (TRL’s `GRPOTrainer`): for every (source poem, target author) prompt the policy samples $G = 4$ poems, each is scored, and the update pushes probability towards the above-average members of the group. Everything in this section is computed from three inputs only — the generated poem, the source poem and the target author. The Gemma output is never a reward target, and no validation or test poem enters the prompts, the reward calibration or any in-training evaluation.

8.1 Why the SFT adapter is merged first

With a LoRA policy, TRL obtains the KL reference by switching the trainable adapter off. Continuing to train the SFT adapter would therefore anchor the KL term to the *untuned* base model — the model that copies its source under a markdown heading (§7). Instead the SFT adapter is folded into the base weights (`merge_sft_adapter.py`) and a fresh LoRA is trained on top. A fresh LoRA starts as the identity ($B = 0$), so the start policy and the KL reference ($\beta = 0.04$) are both exactly the SFT model, and the base weights stay frozen.

8.2 Style reward

The style reward reuses the profile definitions of `author_style_profiles.csv`, computed per poem instead of per author. The per-poem code was extracted into `style_features.py` and `style_profiler.py` now imports it; the refactored profiler’s output is byte-identical to the unmodified script’s, so the profile definitions are unchanged. (The committed profile CSV is itself slightly stale against the current `pos_tagged.csv` — differences in the fourth decimal — and was deliberately left as it is.) Generated poems are tagged with `tagging.tag_poem`, which tags the whole poem with the same corrections and filtering as the corpus build.

Every feature is scored against the target author’s *per-poem* distribution over their train-split poems (`reward_calibration.json`), not against the author mean alone: a single poem’s noun share varies a lot around its author’s average. For a scalar feature x_f ,

$$s_f = \exp\left(-\frac{1}{2}z_f^2\right), \quad z_f = \frac{x_f - \mu_{a,f}}{\max(\sigma_{a,f}, 0.5\sigma_{\text{corpus},f})}, \quad (1)$$

where the floor keeps authors with few or unusually uniform poems from zeroing every small miss. For the POS-bigram and rhyme-ending distributions the cosine c to the author’s aggregate distribution is scored one-sidedly against the author’s own leave-one-out self-similarity (μ_c, σ_c) : $s = 1$ for $c \geq \mu_c$, otherwise $\exp(-\frac{1}{2}((c - \mu_c)/\sigma_c)^2)$. The classifier’s target-author probability is used as it is, and TF-IDF palette hits are divided by the author’s 90th-percentile hit rate on real poems, since the raw rate is about 0.05 even for human text. Scores are averaged within groups, and R_{style} is the weighted mean of the groups:

Group	Features	Weight
POS profile	pct_NOUN/VERB/ADJ/ADV/PROPN	1.0
lexical diversity	MATTR over lemmas (TTR below the 200 window)	1.0
structure	tokens/line, lines, stanzas, lines/stanza	1.0
POS bigrams	bigram distribution	1.0
classifier	clf_target_prob	1.0
rhyme	rhyme-ending distribution	0.5
TF-IDF	palette hits / p90	0.5

Because the reward is computed against the *target* author’s own distribution, a policy that reverts to Блаже Конески for every target is not rewarded for it — the per-author concern raised in §7.

8.3 Content reward

Content is measured against the source poem with a line-level, BERTScore-style F1 over `paraphrase-multilingual-m` embeddings: recall averages each source line’s best cosine match among the generated lines, precision the reverse. Working per line avoids the model’s 128-token truncation and stops a poem that covers only its first stanza from earning full recall. Because every line takes its best match, unrelated poems already score high: the mean F1 over 2,000 random cross-author train pairs is $b = 0.553$. The reward is rescaled so that this level is zero, $R_{\text{content}} = \text{clip}((F_1 - b)/(1 - b), 0, 1)$.

8.4 Gates

Three failure modes are cheaper for the policy than doing the task, and each is gated multiplicatively rather than added as a separate objective:

- *Copying.* The untuned base model’s characteristic output is the source reproduced verbatim (line novelty 0.22, §7), which an embedding content reward would pay out on in full. R_{content} is multiplied by `copy_novelty`, the share of generated lines that are not near-copies: a line of at least three tokens is a copy if at least 75 % of its word types occur in one source line. Containment is used rather than Jaccard because on this corpus’s ~5-token lines a single swapped word gives Jaccard $4/6 = 0.67$, which a Jaccard threshold of 0.7 would let through.
- *Repetition.* R_{style} is multiplied by line uniqueness (0.97 on real poems).
- *Format.* Both rewards are multiplied by `validity = is_cyrillic_no_chat_markup.is_nonempty`. The markup test is deliberately narrower than the reported `no_markup`: bullet and numbered-line patterns fire on 49 of the corpus’s 1,239 real poems (dialogue lines opening with “- ”), and as a hard gate they zeroed one of the fixture cases’ real target poems. Bold, heading and code-fence markup alone catch 186 of the untuned model’s 191 markup outputs and occur in one real poem.

The total is $R = \alpha R_{\text{style}}^g + \beta R_{\text{content}}^g$ with $\alpha = \beta = 0.5$ as a starting point. Under GRPO’s per-group advantage normalisation, which term drives the update depends on its spread *within* a group of samples, not on its mean, so α, β are to be set from `evaluate_reward.py --groups` before the long run.

8.5 Validation before training

`evaluate_reward.py --fixtures` scores seven hand-built candidates for four train-split (source, target) pairs and checks orderings that any usable reward must satisfy. All of them hold:

Candidate	R_{style}	R_{content}	R_{content}^g	R
real poem by the target	0.75–0.90	0.00–0.05	0.00–0.05	0.38–0.45
verbatim source	0.49–0.63	1.00	0.00	0.25–0.32
near-copy (one word per line)	0.43–0.58	0.50–0.72	0.00	0.22–0.29
Gemma teacher output	0.61–0.83	0.00–0.26	0.00–0.26	0.35–0.51
one line repeated 12×	0.19–0.29	0.00–0.37	0.00	0.01
markdown answer in English	0.17–0.36	0.00	0.00	0.00
empty	0.08–0.11	0.00	0.00	0.00

The target author’s own poem gets the highest style score, and both copies lose their entire content reward to the gate. The table also exposes the reward’s main weakness: the Gemma teacher’s raw content score is low (0.00–0.26), so after the baseline rescaling a genuine paraphrase barely clears an unrelated poem. The content term may therefore carry little signal within a group; the `--groups` measurement decides whether it needs a different baseline or model.

At full scale, over the same 468 validation pairs as §7 (`compare_generations.py --rewards`), the reward ranks the SFT model above the untuned base model even though the base model’s raw content score is far higher — the copy and format gates remove it:

Metric	untuned base	SFT
copy novelty	0.197	0.993
$R_{\text{style}} / \text{gated}$	0.487 / 0.272	0.425 / 0.387
$R_{\text{content}} / \text{gated}$	0.817 / 0.074	0.121 / 0.120
R	0.173	0.254
rows with $R = 0$	39.7 %	0.0 %

The SFT model’s content reward is low (0.121, and exactly zero for 19 % of rows), consistent with the teacher’s own weak content preservation.

8.6 Known limitations

- `classifier.pkl` and the TF-IDF palettes were fit on the whole corpus, validation and test poems included. Both are author-level stylometry and never see the target of a transfer, but they are a leak path; retraining them on the train split is a follow-up.
- A paraphrase written in another Cyrillic-script language (Serbian, Russian) passes the validity gate and scores well on multilingual content similarity. This has not been observed in SFT output.
- Below the 200-token MATTR window lexical diversity falls back to TTR, which depends on poem length; z-scoring against the same author’s per-poem values keeps it comparable, but it interacts with the line-count feature.

9 Reproducibility

Figures in §2–§2.5 (the diagnosis) were produced against `skg_final.gexf` and `graph_analytics.csv` as of commit 41bb49e (or later, up to the fix below, on branch `feats-morph-lookup`), and against the pre-switch versions recoverable at commit 5f03983/363009a. Figures in §2.6 (the fix) were produced against `src/graph_analytics.py`’s current working-tree state on that same branch, which restricts Louvain’s input graph to word-mediated edges; commit this change alongside the CSV it produces to keep the two in sync. The Louvain seed-stability sweeps and the tier-frequency measurements were produced by one-off scripts run against the live pipeline rather than checked-in tests, since they characterize algorithmic stability and empirical tier frequency rather than fixed expected behavior.

Figures in §7 come from `data/validation_generations/comparison_summary.csv` and `comparison_rows.csv` (the base-vs-tuned run, `slurm/compare_base_lora.slurm`), and from `data/validation_generations/ablation/` (the decoding ablation, `slurm/decoding_ablation.slurm`). Both are scored with `src/compare_generations.py --include-teacher --corpus-reference`.

The generation CSVs are tracked, because sampled model output cannot be regenerated bit-for-bit on different hardware. The scores can be: rerunning `compare_generations.py` on the tracked CSVs reproduces the tables on CPU. The matched-budget vocabulary counts, the held-out classifier accuracy and the training-length statistics were produced by one-off scripts, like the Louvain sweeps above.

Figures in §8 come from `data/reward_calibration.json` (`src/build_reward_calibration.py, deterministic`), `src/evaluate_reward.py --fixtures`, and `compare_generations.py --rewards` over the tracked validation CSVs; all three run on CPU. The GRPO jobs themselves are `slurm/grpo_smoke.slurm`, `slurm/reward_groups.slurm`, `slurm/train_grpo.slurm` and `slurm/eval_grpo.slurm`; each run writes its arguments, git commit, merge provenance and calibration hash to `models/qwen3-grpo/run_config`